

Weeks 7-11: Depth, Interviews, Closing

Document 5 of 6 — Days 43 to 78 (Oct 16 - Nov 20)

Thirty-six days. You keep coding every one of them, but the purpose shifts from learning to build toward building depth you can defend under questioning. Days 43 to 56 are fully specified with drills. Days 57 to 78 run on a structured rhythm, because from then on employers control part of your calendar.

WEEK 7 — Microservices — Your Differentiator

Oct 16 - Oct 22

This week you split the capstone into two communicating services. Almost no junior candidate has built a real distributed system, and Montreal's banks and insurers run on exactly this architecture.

Day 43 — Friday, Oct 16

Microservice design and service boundaries

45 min Algorithms	Pattern: Mixed timed set — design-flavoured problems Design problems test whether you can pick data structures under constraints, which is the closest algorithm practice gets to real engineering. Solve in this order: 1. LRU Cache (medium) — HashMap plus doubly linked list 2. Design Twitter (medium) 3. Min Stack (medium) — redo cold, 8 minute target
120 min Learn	Monolith versus microservices and, honestly, when each is correct — interviewers respect a candidate who can argue for the monolith. Service boundaries and bounded contexts. Data ownership: why each service owns its database and why a shared database defeats the purpose. Synchronous versus asynchronous communication. The distributed-systems costs you must be able to name: network failure, eventual consistency, distributed transactions, operational complexity.
DRILLS · Service boundaries and independent deployability <i>Goal: write every one of these yourself, by hand, no copy-paste.</i>	
1. Write a one-page design document for the split: an Expense Service and a Notification Service, each with its own database and clear responsibilities. 2. Write down three concrete reasons this split is justified, and two arguments for keeping it a monolith. Be able to argue both sides. 3. Define the contract between the services: what events flow, what synchronous calls exist, and what data each owns. 4. Draw the architecture diagram and photograph it for the README. 5. Scaffold the Notification Service as a separate Maven project with its own package structure, database, and Flyway baseline. 6. Add it to docker-compose with its own PostgreSQL instance on a different port. Confirm both services and both databases start together. 7. Give the Notification Service its own domain: a Notification entity with recipient, type, subject, body, status, sentAt. 8. Implement its internal CRUD and a stub send method that logs rather than actually sending. Test it independently.	
135 min Build	Two independently running services, each with its own database, starting together from one docker-compose command. Commit both repositories.
60 min Job search	Send 8 applications. Prepare for any scheduled calls.

✓ **Win of the day: You designed a distributed system yourself and can argue both for and against the split.**

Day 44 — Saturday, Oct 17

Synchronous service communication

45 min Algorithms	Pattern: Mixed timed set Continue mixed practice, tracking pattern-recognition speed. Solve in this order: 1. Four mediums drawn from graphs and DP 2. One from your repeated-failure list
120 min Learn	Service-to-service HTTP: RestTemplate (legacy), WebClient (reactive, current), and OpenFeign (declarative). Timeouts and why a missing timeout can take down an entire system through thread exhaustion. Connection pooling for outbound calls. Contract design between services and versioning. Handling downstream failure.
DRILLS · Synchronous inter-service calls and failure handling <i>Goal: write every one of these yourself, by hand, no copy-paste.</i>	
1. Add WebClient to the Expense Service and configure a connection timeout and a response timeout explicitly. 2. Implement a call from Expense Service to Notification Service when an expense is approved. 3. Define a shared request DTO for the contract and document it. Note the versioning problem this creates. 4. Test the happy path and confirm the notification is created in the second service's database. 5. Stop the Notification Service and call the approve endpoint. Observe exactly what happens and how long it takes. 6. Configure a 2-second timeout, repeat the test, and confirm the failure is now fast rather than hanging. 7. Decide and implement a policy: should approval fail if notification fails? Write your reasoning in a comment — this is a real design question interviewers ask. 8. Add logging around the outbound call including duration and outcome. 9. Write an integration test using a mock server (WireMock or MockWebServer) covering success, timeout, and 500 from downstream.	
135 min Build	Two services communicating over HTTP with explicit timeouts and tested failure handling. Commit.
60 min Job search	Send 8 applications.

✓ **Win of the day: Two independent services of yours are talking, and you have handled the failure case deliberately rather than by accident.**

Day 45 — Sunday, Oct 18

Asynchronous messaging

45 min Algorithms	Pattern: Mixed timed set Mixed practice. Solve in this order: 1. Four mediums 2. One hard attempted for 30 minutes
120 min Learn	Message brokers: queues versus topics, RabbitMQ concepts (exchange, binding, routing key, queue), delivery guarantees (at-most-once, at-least-once, exactly-once and why the last is hard), idempotent consumers, dead letter queues, and message ordering. When asynchronous beats synchronous and what you give up.
DRILLS · Event-driven messaging with idempotent consumers <i>Goal: write every one of these yourself, by hand, no copy-paste.</i>	
1. Add RabbitMQ to docker-compose and confirm the management UI loads. 2. Define an ExpenseApprovedEvent with the data the Notification Service needs, and publish it on approval. 3. Implement the consumer in the Notification Service and confirm end-to-end flow through the broker. 4. Remove the synchronous call from Day 44 and note in your log the trade-off you just made. 5. Make the consumer idempotent by storing processed event IDs and skipping duplicates. Publish the same event twice and prove only one notification is created. 6. Configure a dead letter queue. Make the consumer throw deliberately, and confirm the failed message lands in the DLQ rather than looping forever. 7. Stop the Notification Service, approve three expenses, restart it, and confirm all three notifications process from the queue. This is the durability benefit — record it in your log. 8. Add retry with a limit before dead-lettering, and explain your chosen limit in a comment. 9. Write an integration test using a Testcontainers RabbitMQ instance.	
135 min Build	An event-driven system with idempotent consumers, retries, and a dead letter queue, all tested. Commit.
60 min Job search	Send 8 applications. Follow up on week-old applications.

✓ **Win of the day: Your system is event-driven with durability guarantees — a genuinely senior-sounding capability at your level.**

Day 46 — Monday, Oct 19

Resilience patterns

45 min Algorithms	Pattern: Mixed timed set Mixed practice. Solve in this order: 1. Four mediums 2. Redo two problems you have now failed twice
120 min Learn	Resilience4j: circuit breaker states (closed, open, half-open) and the thresholds that move between them; retry with exponential backoff and jitter; bulkhead for resource isolation; rate limiter; and fallback methods. Cascading failure and how a circuit breaker stops it. Why retrying a non-idempotent operation is dangerous.
DRILLS · Resilience — circuit breakers, retries, and bulkheads <i>Goal: write every one of these yourself, by hand, no copy-paste.</i>	
1. Add Resilience4j and wrap any remaining synchronous inter-service call in a circuit breaker. 2. Configure the failure-rate threshold, wait duration, and half-open call count deliberately, and explain each choice in a comment. 3. Write a fallback method returning a degraded but useful response. 4. Kill the downstream service and watch the circuit transition from closed to open in the Actuator metrics. Record the observed behaviour. 5. Restart the downstream service and watch the circuit go half-open and then closed. Record the recovery time. 6. Add retry with exponential backoff to a transient-failure path, and write a comment on why you would never retry a non-idempotent POST blindly. 7. Add a rate limiter to your most expensive report endpoint and prove it rejects excess requests with 429. 8. Add a bulkhead limiting concurrent calls to one dependency and explain what problem it isolates. 9. Write a test that forces the circuit open and asserts the fallback response. 10. Screenshot the circuit breaker metrics for your README.	
135 min Build	Circuit breaker, retry, rate limiter, and bulkhead configured and demonstrated. Documented in the README. Commit.
60 min Job search	Send 8 applications.

✓ **Win of the day: You built and demonstrated a circuit breaker protecting a real failure scenario you caused yourself.**

Day 47 — Tuesday, Oct 20

API gateway and centralized concerns

45 min Algorithms	Pattern: Mixed timed set Mixed practice. Solve in this order: 1. Four mediums 2. One hard
120 min Learn	API gateway responsibilities: routing, cross-cutting authentication, rate limiting, request aggregation, and why you do not duplicate auth logic in every service. Spring Cloud Gateway: routes, predicates, filters. Centralized configuration concepts. Service discovery at a conceptual level.
DRILLS · API gateway and centralized authentication <i>Goal: write every one of these yourself, by hand, no copy-paste.</i>	
1. Create a Spring Cloud Gateway project and add it to docker-compose. 2. Configure routes forwarding to the Expense Service and the Notification Service. 3. Move JWT validation into the gateway so downstream services trust an authenticated header. 4. Confirm that calling a service directly, bypassing the gateway, is now blocked at the network level in Docker Compose. 5. Add a global filter logging every request with a correlation ID, and propagate that ID downstream. 6. Add rate limiting at the gateway level and test it. 7. Add a custom filter injecting the authenticated user ID into a header, and consume it downstream. 8. Test the full flow end to end through the gateway: register, log in, create, submit, approve, and confirm the notification arrives. 9. Update your architecture diagram to include the gateway.	
135 min Build	All traffic flows through a single secured entry point. Full flow verified end to end. Commit.
60 min Job search	Send 8 applications.

✓ **Win of the day: Your system has a real production topology: gateway, two services, two databases, and a message broker.**

Day 48 — Wednesday, Oct 21

Distributed tracing and deployment

45 min Algorithms	Pattern: Mixed timed set Mixed practice. Solve in this order: 1. Four mediums, 20 minute target each 2. Track how many you identify correctly within two minutes
120 min Learn	Distributed tracing: traces, spans, and context propagation across service boundaries. Why debugging distributed systems is fundamentally harder than debugging a monolith. Micrometer Tracing basics. Centralized logging concepts. Health checks in a multi-service system.
DRILLS · Distributed tracing across multiple services <i>Goal: write every one of these yourself, by hand, no copy-paste.</i>	
1. Propagate your correlation ID from the gateway through both services and into the message consumer. 2. Trigger one expense approval and produce a single log view showing every line from all three services sharing one ID. 3. Add Micrometer Tracing and observe span creation across the HTTP boundary. 4. Add health checks for every service and a compose-level view of the whole system's health. 5. Deliberately break one service and confirm the gateway health reflects it. 6. Write a runbook section in your README: how to diagnose a failed notification, step by step, using logs and metrics. 7. Bring the entire stack down and up from scratch and time how long until it is fully healthy. 8. Write down the three hardest things about debugging this system versus the monolith — this is interview material.	
135 min Build	Full distributed traceability across three services and a broker, plus a runbook. Commit.
60 min Job search	Send 8 applications.

✓ **Win of the day: You can trace one request across three services — the skill that makes distributed debugging possible at all.**

Day 49 — Thursday, Oct 22

Document and present the system

45 min Algorithms	Pattern: Revisit list clearance Everything that has failed twice comes back today. A problem you finally solve cold after two failures is worth more than five new easy problems. Solve in this order: 1. Six problems from your repeated-failure list 2. Anything still failing goes into daily rotation for Week 8
120 min Learn	Preparing the architecture narrative: why two services, why RabbitMQ for that event, what you would change at ten times the load, and what you would do differently if starting over. Interviewers reward candidates who can criticize their own design.
DRILLS · Architecture documentation and whiteboard fluency <i>Goal: write every one of these yourself, by hand, no copy-paste.</i>	
1. Write the README for the microservices project: problem, architecture diagram, why the split, the event flow, resilience patterns, and how to run it. 2. Write a 'Trade-offs and What I'd Do Differently' section naming at least three honest weaknesses in your design. 3. Record a demo showing the event flow end to end and the circuit breaker opening and recovering. 4. Practise whiteboarding the architecture from memory in under three minutes, twice. 5. Prepare answers to: why not a monolith, why RabbitMQ over Kafka, how do you handle a duplicate event, what happens if the broker goes down, and how would you deploy this to production. 6. Add both projects to your resume with links and one-line stack summaries. 7. Post the project on LinkedIn with the architecture diagram and a short write-up of what you learned. 8. Rehearse the whole walkthrough once in French.	
135 min Build	Both projects fully documented, demoed, and published. Resume and LinkedIn updated.
60 min Job search	Send 8 applications. You should be near 120 cumulatively.

✓ **Win of the day: Two substantial projects on GitHub, one a distributed system you can whiteboard from memory in two languages.**

Pure interview preparation. The build hours go to focused deep-dive exercises rather than product work, because these are the topics Java interviewers actually test.

Day 50 — Friday, Oct 23

Concurrency fundamentals

45 min Algorithms	Pattern: Mixed timed, interview conditions From here, every session is narrated aloud as if an interviewer were listening. Silence reads as being stuck. Solve in this order: 1. Three mediums solved while talking through your reasoning aloud 2. Record one session and watch it back
120 min Learn	Threads and Runnable. The Executor framework, thread pools, and pool sizing. Callable and Future. CompletableFuture and composition. Race conditions, critical sections, synchronized, volatile, and the atomic classes. The Java Memory Model at a working level: visibility, ordering, happens-before. Deadlock, livelock, and starvation. Concurrent collections. Virtual threads at a conceptual level.
DRILLS · Concurrency — race conditions, deadlocks, and parallelism <i>Goal: write every one of these yourself, by hand, no copy-paste.</i>	
1. Write a counter incremented by 10 threads 10,000 times each without synchronization. Run it five times and record the wrong results. 2. Fix it three ways: synchronized, AtomicInteger, and a lock. Verify each produces exactly 100,000. 3. Demonstrate a visibility bug with a non-volatile boolean flag that one thread never sees updated, then fix it with volatile. 4. Write a deliberate deadlock with two locks acquired in opposite orders. Run it, observe the hang, then take a thread dump and find the deadlock in it. 5. Fix the deadlock by ordering lock acquisition consistently. 6. Use an ExecutorService with a fixed pool to run 100 tasks. Experiment with pool sizes of 1, 4, and 50 and record the timings. 7. Use CompletableFuture to run two independent slow operations in parallel, combine the results, and measure the speedup against sequential execution. 8. Apply this to your capstone: parallelize two independent report queries with CompletableFuture and record the before-and-after timings. 9. Compare HashMap and ConcurrentHashMap under concurrent writes and observe the corruption in the first. 10. Write your explanation of volatile versus synchronized in your log, as you would say it in an interview.	
135 min Build	A concurrency drill package in your repo demonstrating every concept above, plus the parallelized report in your capstone.
60 min Job search	Send 8 applications. Prepare specifically for any scheduled interview.

✓ **Win of the day: You caused and fixed a race condition and a deadlock — concurrency is now experience, not theory.**

Day 51 — Saturday, Oct 24

JVM internals and memory

45 min Algorithms	Pattern: Mixed timed, narrated Continue narrated practice. Solve in this order: 1. Three mediums narrated aloud 2. One hard attempted for 30 minutes
120 min Learn	JVM memory areas: heap (young generation, old generation), stack, metaspace, and the code cache. Object allocation and escape analysis. Garbage collection: generational hypothesis, minor versus major collections, stop-the-world pauses, and the main collectors (Serial, Parallel, G1, ZGC) at a working level. Common causes of OutOfMemoryError. Memory leaks in Java despite garbage collection. Reading a heap dump. JVM flags worth knowing.
DRILLS · JVM memory, garbage collection, and leak diagnosis <i>Goal: write every one of these yourself, by hand, no copy-paste.</i>	
1. Write a program that deliberately causes OutOfMemoryError: Java heap space by accumulating objects in a static List. Run it with a small -Xmx and observe the failure. 2. Write a program causing StackOverflowError through unbounded recursion, and note the difference between the two failures. 3. Create a genuine memory leak: register listeners in a static collection and never deregister them. Watch the heap grow in VisualVM or JConsole. 4. Take a heap dump at the point of failure and identify the leaking objects in it. 5. Fix the leak and confirm the heap stabilizes under the same load. 6. Run your capstone with -verbose:gc and observe the collection pattern under load from your load script. 7. Compare G1 and Parallel collectors on the same workload and record pause times for each. 8. Set -Xmx deliberately for your Docker container and explain in a comment why container memory limits and JVM heap must be configured together. 9. Write your explanation of how garbage collection works in your log, in the words you would use out loud.	
135 min Build	A JVM drill package demonstrating each failure mode, and documented memory settings for your containerized capstone.
60 min Job search	Send 8 applications.

✓ **Win of the day: You have watched a JVM leak memory and fixed it from a heap dump — almost no junior candidate has done this.**

Day 52 — Sunday, Oct 25

Spring internals

45 min Algorithms	Pattern: Mixed timed, narrated Continue narrated practice. Solve in this order: 1. Three mediums narrated 2. Two from your failure list
120 min Learn	The bean lifecycle in full: instantiation, population, aware interfaces, BeanPostProcessor, initialization callbacks, use, destruction. AOP: what a proxy is, JDK dynamic proxies versus CGLIB, aspects, pointcuts, advice types. Why self-invocation bypasses @Transactional, @Cacheable, and @Async — the single most common Spring gotcha. Auto-configuration and @Conditional. How @SpringBootApplication decomposes.
DRILLS · Spring internals, AOP, and the self-invocation trap <i>Goal: write every one of these yourself, by hand, no copy-paste.</i>	
1. Write a BeanPostProcessor logging every bean as it is created, and observe the full startup order. 2. Write a custom aspect with @Around that logs execution time of every method in your service package. Verify it works. 3. Extend the aspect to log method arguments and return values, and note what you must never log. 4. Demonstrate the self-invocation trap: call a @Transactional method from another method in the same class, prove no transaction starts, then fix it by extracting to a separate bean. 5. Repeat the same demonstration with @Cacheable and confirm the cache is bypassed identically. 6. Print whether a given bean is a JDK proxy or a CGLIB proxy at runtime, and explain in a comment what determines which is used. 7. Write a @Conditional bean that only loads when a property is set, and prove it appears and disappears with the property. 8. Trace what @SpringBootApplication expands to and write each component annotation in your log with what it does. 9. Write your explanation of how @Transactional actually works in your log, as you would say it aloud.	
135 min Build	An AOP and internals drill package, plus timing aspects applied to your capstone.
60 min Job search	Send 8 applications.

✓ **Win of the day: You can explain how Spring works internally, which separates you from candidates who only use it.**

Day 53 — Monday, Oct 26

System design part one

45 min Algorithms	Pattern: One hard, seriously attempted One hard problem worked properly teaches more today than three mediums. Solve in this order: 1. Trapping Rain Water (hard) or Largest Rectangle in Histogram (hard) 2. Then two mediums narrated
120 min Learn	The system design framework: clarify requirements, estimate scale with real numbers, define the API, design the data model, sketch the high-level architecture, then identify and address bottlenecks. Load balancing. Horizontal versus vertical scaling. Stateless services and why they scale. Back-of-envelope estimation.
DRILLS · The system design framework and back-of-envelope estimation <i>Goal: write every one of these yourself, by hand, no copy-paste.</i>	
1. Learn the six-step framework by heart and write it in your log. 2. Practise estimation: how much storage for 10 million expenses a year, how many requests per second at 100,000 daily active users, how much bandwidth. Show your arithmetic. 3. Design a URL shortener in 40 minutes, out loud, on paper, with a timer. Cover all six steps. 4. Write up your design afterward and compare it against a reference design. List what you missed. 5. Design a rate limiter in 40 minutes with the same discipline, covering at least two algorithm options and their trade-offs. 6. Write up and compare again. 7. For both designs, name the single biggest bottleneck and how you would address it. 8. Practise explaining your capstone's architecture using the same six-step framework.	
135 min Build	Two complete written system designs with self-assessed gap lists.
60 min Job search	Send 8 applications.

✓ **Win of the day: You completed two full system design exercises under timed interview conditions.**

Day 54 — Tuesday, Oct 27

System design part two

45 min Algorithms	Pattern: One hard, then narrated mediums Same structure as yesterday. Solve in this order: 1. One hard 2. Two mediums narrated
120 min Learn	Databases at scale: replication (leader-follower, replication lag), sharding strategies and their pain, read replicas, connection limits. CAP theorem stated correctly. SQL versus NoSQL trade-offs argued honestly. Caching layers and cache invalidation at scale. CDNs. Message queues in architecture. Eventual consistency and what it means for a user.
DRILLS · Scaling databases and reasoning about distributed data <i>Goal: write every one of these yourself, by hand, no copy-paste.</i>	
1. Design 'scale my Expense API to one million users' in 45 minutes, out loud, with a timer. 2. Identify every bottleneck in your current capstone architecture and write the fix for each. 3. Design a notification system handling 10 million messages a day, covering fan-out, delivery guarantees, and retry. 4. For one design, explicitly address what happens when the database becomes the bottleneck: replicas, then sharding, and the shard key you would choose. 5. Explain CAP in your own words in your log, and identify what your capstone chooses and why. 6. Write out an honest comparison of PostgreSQL versus a document store for your expense domain, arguing both sides. 7. Practise the question 'what would you change about your project if it had a million users' until the answer is fluent — you will be asked it. 8. Compare all three of your designs against reference material and record the recurring gaps.	
135 min Build	Two more written designs plus a documented scaling analysis of your own capstone.
60 min Job search	Send 8 applications.

✓ **Win of the day: You can reason credibly about scale, which is what separates intermediate from junior in interviews.**

Day 55 — Wednesday, Oct 28

Debugging and take-home simulation

45 min Algorithms	Pattern: Mixed timed Mixed practice. Solve in this order: 1. Four mediums 2. One from your failure list
120 min Learn	Approaching an unfamiliar codebase: find the entry points, trace one request end to end, then form hypotheses before changing anything. Debugger technique: breakpoints, conditional breakpoints, expression evaluation, stepping strategies. Reading someone else's stack trace. What take-home graders actually look for.
DRILLS · Navigating unfamiliar code and take-home technique <i>Goal: write every one of these yourself, by hand, no copy-paste.</i>	
1. Clone an open-source Spring Boot project you have never seen. Spend 30 minutes mapping its architecture without running it. 2. Run it, trace one request end to end with a debugger, and document the path through the layers. 3. Add one small feature to it in 60 minutes, with a test. This simulates a real take-home. 4. Write down what made the codebase easy or hard to work in, and apply one lesson to your own project. 5. Practise conditional breakpoints: set one that only triggers on a specific input value in your capstone. 6. Deliberately introduce three bugs into your capstone — an off-by-one, a null handling error, and a transaction boundary mistake. Have them sit overnight. 7. Write a take-home checklist for yourself: tests, README, commit hygiene, error handling, and a note on what you would add with more time. 8. Time yourself building a minimal CRUD API from an empty Spring Initializr project. Target under 90 minutes.	
135 min Build	A feature contributed to an unfamiliar codebase, plus your personal take-home checklist.
60 min Job search	Send 8 applications.

✓ **Win of the day: You proved you can be productive in a codebase you did not write — exactly what a take-home tests.**

Day 56 — Thursday, Oct 29

Full mock interview loop

45 min Algorithms	Pattern: Interview conditions Today the algorithm block is part of the mock loop rather than separate. Solve in this order: 1. Two mediums under strict 25-minute limits, narrated, recorded
120 min Learn	Review Document 6, Section 7. Rehearse your six STAR stories out loud in both languages until they run under two minutes each without notes.
DRILLS · A full mock interview loop under real conditions <i>Goal: write every one of these yourself, by hand, no copy-paste.</i>	
1. Find the three bugs you planted on Day 55 using only tests and the debugger. Time yourself. 2. Run a full recorded mock loop: 45 minutes coding, 45 minutes Java and Spring questions, 30 minutes system design, 30 minutes behavioural. 3. For the Java and Spring section, answer aloud: explain dependency injection; interface versus abstract class; what is the N+1 problem; how does @Transactional work; volatile versus synchronized; how does garbage collection work; what happens when you call save on a JPA entity. 4. Watch the recording back and write three specific, actionable fixes. 5. Re-record the weakest segment until it is fluent. 6. Rehearse your capstone walkthrough and your microservices walkthrough back to back. 7. Run the behavioural section once entirely in French. 8. Write your answer to 'why are you moving from QA to backend development' and rehearse it until it sounds confident rather than apologetic.	
135 min Build	A recorded full mock loop with a written self-assessment and three fixes applied.
60 min Job search	Send 8 applications. Push hard on any process that has gone quiet.

✓ **Win of the day: You have been through a complete interview loop already, so the real one will not be your first.**

Days 57 to 78. Employers now control part of your calendar, so these weeks run on a fixed daily rhythm with rotating focus rather than unique daily topics. **A scheduled interview or a take-home always takes priority over that day's plan.**

Block	Time	What you do every day
Algorithms	60 min	2 problems: one new medium, one from your failure list. Narrated aloud, timed at 25 minutes each.
Interview prep	90 min	Rotates on a 5-day cycle: (1) Java and Spring question drills, (2) system design exercise, (3) behavioural rehearsal in
Build	90 min	Keep coding daily. Priority order: (1) any take-home assignment, (2) fixes to weaknesses exposed while presenting yo
Job search	120 min	Applications, follow-ups, recruiter calls, scheduling, and negotiation. See Document 6.

Week 9 — Days 57 to 63 (Oct 30 - Nov 5): Volume and first conversions

- First-round screens and technical interviews should start landing this week — applications sent in early October mature around now.
- Before every interview: 30 minutes of company research, re-read the posting, map your two projects onto their stack, prepare three questions to ask them.
- After every interview, write down every question you were asked. By Week 10 this becomes a study list built from real data instead of guesswork.
- Keep applying at 8 per day. Most processes will not convert, and volume is your protection.
- Build block this week: fix every weakness that surfaced while presenting your projects in real interviews.
- Milestone: 160+ applications cumulatively, at least 4 live processes by November 5.

Week 10 — Days 64 to 70 (Nov 6 - Nov 12): Convert

- Interview prep now targets your actual pipeline: study the specific stack of each company you are interviewing with, not generic material.
- Take-homes take top priority and a 48-hour turnaround. Over-deliver on tests, README, and commit hygiene — these are graded more heavily than the feature.
- Run at least two live mock interviews with a real person this week, not just recordings.
- Start signalling timeline politely: mention you are working toward a late-November decision point.
- Keep applying at 5 per day. Do not stop while interviewing.
- Milestone: multiple final-round processes in flight.

Week 11 — Days 71 to 78 (Nov 13 - Nov 20): Close

- Push every live process forward directly and politely. Ask whether final rounds can be accelerated.
- If an offer arrives, get it in writing and take 24 to 48 hours before responding, even if you intend to accept.
- If multiple processes are live, tell the others you have an offer in hand. Nothing compresses timelines more reliably.
- Negotiate per Document 6, Section 9. If base salary is fixed, negotiate start date, vacation, or a relocation allowance instead.

- Keep coding at least one hour a day through this week. It keeps you sharp for final technical rounds and steadies your nerves.
- **Day 78 — Friday November 20:** the target. A signed offer for a Java backend role in Montreal.

If November 20 arrives with processes still live

Decide this now, calmly, rather than under pressure in November. Enterprise hiring in Quebec commonly runs six to eight weeks end to end, so applications sent in late October may conclude in early December regardless of how well you perform. If that happens, keep the pipeline running rather than abandoning it in week twelve — which is exactly when your October applications start converting. Nothing you built in these 78 days expires on November 20.